

DL Assignment — Fashion MNIST Classification using CNN

I analyzed your complete Fashion MNIST practical PDF carefully.

This practical is MUCH more advanced than normal basic DL practicals because it uses:

- CNN (Convolutional Neural Network)
- BatchNormalization
- Dropout
- Validation split
- EarlyStopping
- ReduceLROnPlateau
- Confusion Matrix
- Prediction visualization

So I'll explain EVERYTHING in simple easy language.

1. What Is Fashion MNIST?

Fashion MNIST is a dataset containing:

- clothing images
- grayscale images
- 28×28 pixels
- 10 categories/classes

Examples:

Class Number Clothing Type

0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

Dataset images shown in your PDF are sample clothing images from these classes.

2. Main Goal of This Assignment

We need to:

1. load Fashion MNIST dataset
2. preprocess image data
3. build CNN model
4. train deep learning model
5. classify clothing images
6. evaluate prediction accuracy

3. What Type of Problem Is This?

This is:

Multi-Class Classification Problem

because:

- output belongs to one class among 10 classes

Example:

- shirt
- shoe
- bag

NOT continuous numerical values.

4. Overall Workflow

Load Fashion MNIST Dataset



Normalize Images



Reshape Images



Convert Labels to Categorical



Create CNN Model



Compile Model



Train Model



Evaluate Accuracy



Predict Clothing Classes



Generate Confusion Matrix

5. Library Installation

Code:

```
!pip install tensorflow numpy matplotlib
```

Purpose:

Library	Purpose
TensorFlow	Deep Learning
NumPy	Numerical operations
Matplotlib	Graph plotting

6. Import Libraries

Code imports:

- TensorFlow
- NumPy
- matplotlib
- keras.layers/models
- warnings

Purpose:

Library	Purpose
tensorflow	neural networks
numpy	arrays/matrices
matplotlib	graphs/images
keras.layers	CNN layers
warnings	hide warnings

7. Random Seed

Code:

```
np.random.seed(42)
tf.random.set_seed(42)
```

Purpose:

- reproducible results
- same random initialization
- stable outputs

8. Loading Dataset

Code:

```
tf.keras.datasets.fashion_mnist.load_data()
```

Dataset contains:

Dataset Images

Training 60,000

Testing 10,000

9. Image Normalization

Code:

```
X_train = X_train.astype('float32') / 255.0
```



MOST IMPORTANT CONCEPT

Images contain pixel values:

0 to 255

Where:

- 0 = black
- 255 = white

Normalization converts:

$$x_{\text{normalized}} = \frac{x}{255}$$

Range becomes:

0 to 1

Why needed?

- faster learning
- stable gradients
- better accuracy

10. Reshaping Images

Code:

```
X_train.reshape(-1, 28, 28, 1)
```

Meaning:

Dimension Meaning

-1	automatic image count
28	image height
28	image width
1	grayscale channel

CNN requires channel dimension.

11. One-Hot Encoding

Code:

```
tf.keras.utils.to_categorical(y_train, 10)
```

Converts:

3

into:

```
[0 0 1 0 0 0 0 0]
```

Why?

Because neural network output layer has:

- 10 neurons
- one neuron per class

12. Validation Split

Code creates:

- training set
- validation set

Validation set used during training to:

- monitor overfitting
- check generalization

Shapes shown in PDF:

Dataset Shape

Training (55000, 28, 28, 1)

Validation (5000, 28, 28, 1)

Testing (10000, 28, 28, 1)

13. Sample Images Visualization

The images shown on page 5 display clothing samples from different classes.

Purpose:

- visualize dataset
- understand class labels
- inspect image quality

Example images:

- boots
- dress
- coat
- shirt
- sneaker

14. CNN Model

This is MOST IMPORTANT PART.

Your model uses:

Convolutional Neural Network (CNN)

CNN is specially designed for:

- image processing
- feature extraction
- pattern recognition

15. First Convolutional Block

Code:

Conv2D(32, (3,3))

Meaning:

- 32 filters
- kernel size = 3×3

Purpose:

- detect patterns
- edges
- textures
- shapes

16. What Is Convolution?

CNN slides filters over image.

Example:

- edge detection
- corners
- curves
- clothing patterns

Filter learns important features automatically.

17. ReLU Activation

Code:

activation='relu'

Formula:

$\text{ReLU}(x) = \max(0, x)$

Purpose:

- removes negative values
- introduces non-linearity
- improves learning

18. BatchNormalization

Code:

BatchNormalization()

Purpose:

- stabilizes training
- speeds learning
- prevents exploding/vanishing gradients

19. MaxPooling

Code:

MaxPooling2D((2,2))

Purpose:

- reduces image size
- reduces computation
- keeps important features

Example:

$28 \times 28 \rightarrow 14 \times 14$

20. Dropout

Code:

Dropout(0.25)

Purpose:

- prevents overfitting
- randomly disables neurons during training

Helps model generalize better.

21. Second and Third Convolution Blocks

Your model increases filters:

- $32 \rightarrow 64 \rightarrow 128$

Why?

Because deeper layers learn:

- more complex patterns
- clothing structure
- advanced visual features

22. Flatten Layer

Code:

Flatten()

Converts:

3D feature maps $\rightarrow$ 1D vector

Required before Dense layer.

23. Dense Layer

Code:

```
Dense(512)
```

Purpose:

- learns high-level patterns
- performs classification

24. Output Layer

Code:

```
Dense(10, activation='softmax')
```

Why 10 neurons?

Because:

- 10 clothing classes

25. Softmax Activation

Softmax converts outputs into probabilities.

Formula:

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Output example:

[0.01, 0.02, 0.90, ...]

Highest probability = predicted class.

26. Model Summary

Model summary shown in PDF explains:

- output shape
- parameter count
- layer types

Total parameters:

885,738

These are learnable weights.

27. Compile Model

Code:

```
optimizer='adam'
```

```
loss='categorical_crossentropy'
```

```
metrics=['accuracy']
```

Adam Optimizer

Controls:

- weight updates
- learning process

Very efficient optimizer.

Categorical Crossentropy

Used for:

- multi-class classification

Formula:

$$L = -\sum y_i \log(\hat{y}_i)$$

Lower loss = better model.

28. EarlyStopping

Code:

```
EarlyStopping()
```

Purpose:

- stops training automatically

- prevents overfitting
- restores best weights

29. ReduceLROnPlateau

Purpose:

- reduces learning rate if improvement stops
- helps fine tuning

30. Training Process

Epoch results shown in PDF:

Important observations:

- accuracy increases
- loss decreases
- validation accuracy improves

Final validation accuracy around:

93%

Very good performance.

31. Accuracy Graph

Graph on page 12 shows:

- training accuracy
- validation accuracy

Interpretation:

- curves increasing = learning successful
- close curves = less overfitting

32. Loss Graph

Loss graph shows:

- training loss decreasing
- validation loss decreasing

Good sign:

- model learning correctly

33. Sample Predictions

Prediction images shown on pages 13–14 display:

- actual label
- predicted label
- correct predictions in green

Purpose:

- visualize model performance
- check real predictions

34. Test Accuracy

Final accuracy:

0.9337

Meaning:

93.37% accuracy

Excellent model performance.

35. Confusion Matrix

MOST IMPORTANT IMAGE.

Confusion matrix shown on last page explains:

- correct predictions
- incorrect predictions
- class confusion

Understanding Confusion Matrix

Rows:

- actual labels

Columns:

- predicted labels

Diagonal values:

- correct predictions

Example:

- large diagonal numbers = good model

Example Interpretation

If:

- Shirt predicted as Coat

then confusion appears off-diagonal.

This shows:

- which classes look similar

36. Why CNN Works Better Than Normal Neural Network

CNN:

- captures spatial patterns
- detects visual features
- uses filters/kernels

Better for images than:

- simple dense networks

37. Time Complexity

CNN complexity depends on:

- filters
- image size
- layers
- epochs

Training is computationally expensive.

38. Important Concepts Used

Concept	Purpose
CNN	image classification
Convolution	feature extraction
Pooling	dimensionality reduction
Dropout	prevent overfitting
BatchNormalization	stabilize training
Softmax	probability output
Crossentropy	classification loss

39. Important Viva Questions

Basic Viva

Q1. What is Fashion MNIST?

Dataset containing clothing images.

Q2. What type of problem is this?

Multi-class classification problem.

Q3. Why CNN used?

Because CNN is specialized for image processing.

Q4. Why normalize images?

To scale pixel values between 0 and 1.

Q5. Why reshape images?

CNN requires channel dimension.

Q6. Why use one-hot encoding?

To represent categorical labels numerically.

Q7. Why output layer has 10 neurons?

Because dataset has 10 classes.

Q8. Why softmax used?

To generate class probabilities.

Q9. What is convolution?

Operation that extracts image features using filters.

Q10. What is pooling?

Technique to reduce image dimensions.

Q11. Why dropout used?

To prevent overfitting.

Q12. What is BatchNormalization?

Technique to stabilize and speed training.

Q13. What does ReLU do?

Introduces non-linearity and removes negative values.

Q14. What is overfitting?

Model memorizes training data and performs poorly on new data.

Q15. What is validation data?

Data used to evaluate model during training.

Q16. What is confusion matrix?

Table showing prediction performance for each class.

Q17. Why categorical_crossentropy used?

Because problem is multi-class classification.

Q18. Difference between training and testing?

Training learns patterns, testing evaluates performance.

Q19. Why CNN better than ANN for images?

CNN preserves spatial image features.

Q20. What does accuracy mean?

Percentage of correct predictions.

40. Short Practical Explanation

This program performs Fashion MNIST image classification using a Convolutional Neural Network (CNN) in TensorFlow/Keras. Images are normalized, reshaped, and converted into categorical labels before training. The CNN model uses convolutional layers, pooling, BatchNormalization, and Dropout for feature extraction and regularization. The model is trained using categorical crossentropy loss and Adam optimizer. Performance is evaluated using accuracy graphs, sample predictions, and confusion matrix visualization.

